CONCEPTUAL QUESTIONS

1. Explain the difference between a virtual machine and a container. Include one use case where each may be preferable.

A virtual machine (VM) virtualizes hardware and runs its own guest operating system on top of a hypervisor. A container virtualizes the application environment by isolating processes while sharing the host operating system's kernel.

Operating System - Virtual Machine has its own guest OS and Container shares the host kernel

Size – Virtual Machine is usually larger than container

Isolation – Virtual Machine has strong Isolation at the VM boundary whereas container has Process Level Isolation

Use cases:

VM: Running a Windows application on a Linux host when a separate Windows operating system is required.

Container: Deploying a Node.js web application that needs to start quickly, scale easily, and run consistently across environments.

2. Explain the roles of Docker Engine, Dockerfile, image, container, volume, and network.

- Docker Engine: The software responsible for building and running containers. It includes the Docker daemon, which manages Docker objects, and interfaces used by Docker clients.

- Dockerfile: A text file containing instructions to build a Docker image, such as selecting a base image, copying files, and installing dependencies.
- Image: An immutable, layered template containing the application code, dependencies, and configuration needed to create containers.
- Container: A running or stopped instance of an image, with its own writable layer and isolated execution environment.
- Volume: A mechanism for storing data independently of a container's lifecycle, allowing data to persist even when the container is removed.
- Network: A mechanism that allows containers to communicate with each other and, when configured, with external systems.

3. Describe what happens when you execute docker run hello-world, from image lookup to container execution.

When the command docker run hello-world is executed:

1. The Docker CLI sends the request to the Docker daemon.
2. Docker checks whether the hello-world image is available locally.
3. If the image is missing, Docker downloads it from the configured registry, typically Docker Hub.
4. Docker creates a container from the image.
5. Docker configures the container's isolated execution environment.
6. Docker starts the container and executes its configured command.
7. The program prints a confirmation message explaining that Docker is working.
8. The program exits, and the container stops. The stopped container remains until it is explicitly removed, unless an automatic removal option was used.

4. Why are containers useful for a function-execution or cloud-agent platform? Mention isolation, reproducibility, and resource considerations.

Containers are useful for function-execution and cloud-agent platforms because they provide a controlled environment for running different tasks.

- Isolation: Each task can execute in its own container, reducing interference between tasks and limiting access to the host.
- Reproducibility: Images package the required runtime, libraries, and application code, helping tasks behave consistently across environments.
- Resource efficiency: Containers generally use fewer resources and start faster than virtual machines, making them suitable for executing many short-lived tasks.
- Resource limits: CPU and memory limits can prevent one task from consuming excessive resources and affecting others.
- Scalability: Containers can be created and removed as demand changes.

5. Compare docker stop, docker kill, and docker rm.

docker stop: Gracefully stops a container by sending a termination signal and waiting for a specified period before force-stopping it.

docker kill: Forcefully stops a container by sending SIGKILL on Linux by default, without waiting for a graceful shutdown period.

docker rm: Removes a container from Docker. A running container must normally be stopped before it can be removed.

6. What is the difference between a Docker image layer and a running container's writable layer?

A Docker image consists of read-only layers that store application files, dependencies, and other image contents. These layers can be shared by multiple containers.

When a container is created, Docker adds a separate writable layer on top of the image layers.

- Image layers: Read-only and shared where possible between containers created from the same image.
- Writable layer: Stores changes made inside that particular container, such as modified files and newly created files.
- Lifecycle: Changes in the writable layer are lost when the container is removed, unless the data was stored separately using a volume or bind mount.

7. Explain FROM, WORKDIR, COPY, RUN, CMD, and EXPOSE in a Dockerfile.

- FROM: Specifies the base image from which the new image is built.
- WORKDIR: Sets the working directory for subsequent instructions and the default working directory for commands executed in the container.
- COPY: Copies files or directories from the build context into the image.
- RUN: Executes a command during image building, commonly to install dependencies.
- CMD: Specifies the default command or arguments executed when a container starts. It can be overridden at runtime.
- EXPOSE: Documents the port on which the application is expected to listen. It does not publish the port to the host.

8. What does .dockerignore do? Give at least four entries that would make sense for a Node.js project and explain why.

The .dockerignore file specifies files and directories that Docker should exclude from the build context. This reduces unnecessary data transfer, speeds up builds, and helps prevent sensitive or irrelevant files from entering the image.

node_modules/: Prevents local dependencies from being copied into the image, where they may be incompatible with the container environment. Dependencies can instead be installed during the build.

.git/: Excludes Git history and repository metadata that the application normally does not need.

.env: Prevents environment files that may contain passwords, API keys, or other secrets from entering the build context.

npm-debug.log*: Excludes unnecessary debugging logs.

9. Differentiate publishing a port with -p from declaring EXPOSE in a Dockerfile.

EXPOSE documents a container port, whereas -p publishes a port by mapping a host port to a container port.

10. Why should application containers avoid storing important data only in their writable layer? Explain volumes and bind mounts.

A container's writable layer is tied to that container's lifecycle. If the container is removed, data stored only in that layer is lost. Therefore, important data should be stored separately.

Docker provides two common options:

- Volume: Managed by Docker and stored outside the container's writable layer. Volumes are suitable for persistent application data, such as database files.
- Bind mount: Maps a specific file or directory on the host into the container. It is useful for development, configuration files, and accessing host files.

11. What is the purpose of environment variables in containerized applications? Why should passwords and secret keys not be hard-coded into an image?

Environment variables allow applications to receive configuration values at runtime without embedding them directly in application code or the image.

They can be used for:

- Database hostnames and ports.
- Application modes, such as development or production.
- API endpoints and feature flags.
- Configuration values that differ between environments.

Passwords and secret keys should not be hard-coded into an image because:

1. Anyone who can access the image may be able to inspect its contents or build history.
2. Secrets embedded in images are difficult to rotate safely.
3. The same image may be distributed across multiple environments, unnecessarily exposing credentials.
4. Secrets committed to source control can remain in repository history even after being deleted from the current code.

Secrets should be supplied at runtime through an appropriate secrets manager or secure deployment mechanism. Environment variables are

useful for ordinary configuration, but sensitive values require care because they may be exposed through process inspection, logs, or debugging tools.

12. What are container health checks and restart policies? Explain when each is useful.

Health checks determine whether an application inside a container is functioning correctly. They run a configured test, such as an HTTP request to a health endpoint, and report a status such as starting, healthy, or unhealthy.

They are useful when a container's process is running but the application may not be responding correctly. An unhealthy status does not, by itself, automatically restart a container.

Restart policies specify whether Docker should restart a container when its main process exits.

Common policies include:

- no: Do not automatically restart the container.
- on-failure: Restart when the container exits with a non-zero exit code.
- always: Restart the container whenever it stops, subject to Docker's restart behaviour.
- unless-stopped: Restart it unless it was explicitly stopped.